

Rexell Manual Testing Guide & Test Cases

This guide provides step-by-step test cases to manually verify all core modules on the Rexell Web3 ticketing platform.

■ ■ Prerequisites & Setup

Ensure the following components are ready before starting manual tests:

1. **Wallets:** Two Metamask or Valora wallets configured on **Celo Sepolia Testnet**.
 - **Wallet A (Organizer / Admin):** Needs test CELO for gas.
 - **Wallet B (Buyer / Seller):** Needs test CELO for gas and cUSD for purchase.
 - **Wallet C (Second Buyer):** Needs test CELO and cUSD.
 2. **Test Tokens:**
 - **CELO Faucet:** [Celo Sepolia Faucet](https://faucet.celo.org/Sepolia)
 - **cUSD:** Obtain via the Faucet or swap CELO to cUSD on Ubeswap/Mento.
 3. **Application:** The frontend server must be running (`npm run dev` in `/frontend`) and the smart contracts deployed.
-

■ Module 1: Soulbound KYC Identity (SoulboundIdentity.sol)

Verify non-transferable KYC and trust scoring on-chain.

Test Case 1.1: Admin Minting Identity

- **Goal:** Verify admin can mint a unique identity NFT linked to a KYC score.
- **Steps:**
 1. Connect **Wallet A (Admin)** to the contract interaction interface.
 2. Call `mintIdentity(userAddress, score)` where `userAddress` is **Wallet B** and `score` is 80.
 3. Confirm the transaction on the block explorer.
- **Expected Result:** ■ Identity token ID 1 is minted to Wallet B. Wallet B's verification score is recorded as 80.

Test Case 1.2: Prevent Non-Admin Minting

- **Goal:** Verify access control on identity creation.
- **Steps:**
 1. Connect **Wallet B (non-admin)**.
 2. Attempt to call `mintIdentity(Wallet C, 90)`.
- **Expected Result:** ■ The transaction must fail/revert with "Ownable: caller is not the owner".

Test Case 1.3: Prevent Duplicate Identity Mints

- **Goal:** Enforce single identity per wallet constraint.
- **Steps:**

1. Connect **Wallet A (Admin)**.
 2. Attempt to call `mintIdentity(Wallet B, 85)` again.
- **Expected Result:** ■ The transaction must fail/revert with "User already has an identity".

Test Case 1.4: Update Identity Score

- **Goal:** Verify admin can modify user trust scores dynamically.
- **Steps:**
 1. Connect **Wallet A (Admin)**.
 2. Call `updateScore(Wallet B, 65)`.
 3. Query `hasValidIdentity(Wallet B)`.
- **Expected Result:**
 - ■ Score successfully updated.
 - ■ `hasValidIdentity(Wallet B)` returns false (since score 65 is below the KYC threshold of 70).

Test Case 1.5: Soulbound Property Verification

- **Goal:** Confirm the identity token is non-transferable.
 - **Steps:**
 1. Connect **Wallet B** (holder of Identity ID 1).
 2. Attempt to transfer Token ID 1 to **Wallet C** using `transferFrom` or `safeTransferFrom`.
 - **Expected Result:** ■ Transaction reverts with "Soulbound: Transfer not allowed".
-

■ Module 2: Ticketing & Primary Sales (Rexell.sol)

Verify event creation, standard sales, and purchase constraints.

Test Case 2.1: Event Creation

- **Goal:** Verify organizer can set up events with pricing and inventory.
- **Steps:**
 1. Connect **Wallet A (Organizer)**.
 2. Navigate to "Create Event" page (or write on contract).
 3. Fill in details: Event date set to **5 days in the future**, ticket price = 10 cUSD, inventory = 10.
 4. Submit transaction.
- **Expected Result:** ■ Event is successfully registered on-chain with Event ID 0.

Test Case 2.2: KYC Verification Gate (Blocked Purchase)

- **Goal:** Rejects purchasers who have not completed KYC or have a score < 70.
- **Steps:**
 1. Connect **Wallet C** (no Soulbound Identity minted, or score < 70).
 2. Attempt to purchase a ticket for Event ID 0 using `buyTicket`.
- **Expected Result:** ■ Transaction reverts with "Buyer not verified via Soulbound Identity".

Test Case 2.3: KYC Verification Gate (Successful Purchase)

- **Goal:** Permits purchasers with a valid KYC score (≥ 70).
- **Steps:**
 1. Connect **Wallet A (Admin)** and mint an identity to **Wallet B** with score 85.
 2. Connect **Wallet B** (valid KYC).
 3. Verify Wallet B has approved 10 cUSD for the Rexell contract.
 4. Call `buyTicket(0, "ipfs://metadata-hash")`.
- **Expected Result:**
 - ■ Transaction completes successfully.
 - ■ Ticket NFT ID 0 is minted to Wallet B.
 - ■ Event ticket inventory drops to 9.
 - ■ 10 cUSD is transferred from Wallet B to Wallet A.

Test Case 2.4: 4-Ticket Anti-Scalping Purchase Cap (Single Tx)

- **Goal:** Enforce max limit of 4 tickets per transaction.
- **Steps:**
 1. Connect **Wallet B** (valid KYC).
 2. Attempt to call `buyTickets(0, [uri1, uri2, uri3, uri4, uri5], 5)`.
- **Expected Result:** ■ Transaction reverts with "Purchase exceeds 4 tickets limit per user".

Test Case 2.5: 4-Ticket Anti-Scalping Purchase Cap (Cumulative)

- **Goal:** Prevent users from accumulating more than 4 tickets across separate transactions.
- **Steps:**
 1. Connect **Wallet B** (already owns 1 ticket from Test 2.3).
 2. Attempt to buy 4 more tickets: `buyTickets(0, [uri1, uri2, uri3, uri4], 4)`.
- **Expected Result:**
 - ■ Transaction reverts with "Purchase exceeds 4 tickets limit per user".
 - ■ Attempting to buy 3 more tickets (total = 4) succeeds.

Test Case 2.6: Organizer Purchase Prevention

- **Goal:** Prevent organizers from purchasing tickets to their own event.
- **Steps:**
 1. Connect **Wallet A (Organizer)** (holds valid KYC identity).
 2. Attempt to purchase a ticket for Event ID 0 using `buyTicket` or `buyTickets`.
- **Expected Result:** ■ Transaction reverts with "Organizer cannot buy tickets for their own event".

■ Module 3: Seat Mapping & Locks (Rexell.sol)

Verify category pricing, locking states, and double-booking protection.

Test Case 3.1: Set Seat Category Prices

- **Goal:** Set customized pricing per category.

- **Steps:**
 1. Connect **Wallet A (Organizer)**.
 2. Set category price: `setSeatCategoryPrice(0, "VIP", 50 cUSD)`.
 3. Query `seatCategoryPrices(0, "VIP")`.
- **Expected Result:** ■ Returns 50000000000000000000 (50 cUSD with 18 decimals).

Test Case 3.2: Seat Locking Mechanics

- **Goal:** Lock seats to prevent purchase interference.
- **Steps:**
 1. Connect **Wallet B** (valid KYC).
 2. Call `lockSeats(0, ["VIP-A1"])`.
 3. Verify lock is registered.
- **Expected Result:** ■ Seat lock record shows locked by Wallet B, valid for 10 minutes.

Test Case 3.3: Block Concurrent Seat Locking (Race Condition)

- **Goal:** Prevent other users from locking already locked seats.
- **Steps:**
 1. Connect **Wallet C** (valid KYC).
 2. Attempt to lock "VIP-A1" while Wallet B's lock is active.
- **Expected Result:** ■ Transaction reverts with "Seat locked by another user".

Test Case 3.4: Lock Timeout & Release

- **Goal:** Verify locks expire and release automatically.
 - **Steps:**
 1. Wait 10 minutes for Wallet B's lock on "VIP-A1" to expire.
 2. Connect **Wallet C** (valid KYC).
 3. Attempt to lock "VIP-A1".
 - **Expected Result:** ■ Lock succeeds for Wallet C; lock owner is updated.
-

■ Module 4: Resale Marketplace (Rexell.sol)

Verify listing rules, organizer controls, and royalty splits.

Test Case 4.1: Max Price Resale Cap (200%)

- **Goal:** Block listings exceeding 200% of original price.
- **Steps:**
 1. Connect **Wallet B** (owns ticket purchased for 10 cUSD).
 2. Attempt to request resale verification for price 21 cUSD using `requestResaleVerification(tokenId, price)`.
- **Expected Result:** ■ Transaction reverts with "Price exceeds maximum allowed resale price".

Test Case 4.2: Pre-Event Resale Listing Freeze (48h Freeze)

- **Goal:** Prevent listings within 48 hours of the event start time.

- **Steps:**
 1. Connect **Wallet A (Organizer)** and create an event starting in **24 hours**.
 2. Connect **Wallet B** (valid KYC), buy a ticket, and attempt to list it for resale.
- **Expected Result:** ■ Transaction reverts with "Resale period has ended".

Test Case 4.3: Resale Request and Approval Flow

- **Goal:** Verify organizer can review and approve a resale listing.
- **Steps:**
 1. Connect **Wallet B** (owns ticket, event is in 5 days).
 2. Submit `requestResaleVerification(tokenId, 15 cUSD)`.
 3. Connect **Wallet A (Organizer)**.
 4. Call `approveResale(tokenId)`.
 5. Query `resaleRequests(tokenId)`.
- **Expected Result:** ■ Request status is updated to `approved = true`.

Test Case 4.4: Resale Payout splits (Royalty Math)

- **Goal:** Validate split distribution (5% Organizer, 2% Platform, 93% Seller).
- **Steps:**
 1. Connect **Wallet B (Seller)** and approve resale: `resellTicket(tokenId, 20 cUSD, uri)`.
 2. Connect **Wallet C (Buyer)**, ensure cUSD approved.
 3. Record cUSD balances of **Wallet A (Organizer)**, **Wallet B (Seller)**, and **Wallet Fee Recipient**.
 4. Call `buyResaleTicket(tokenId, 20 cUSD)`.
 5. Query updated balances.
- **Expected Result:**
 - ■ Wallet A (Organizer) balance increases by 1.0 cUSD (5%).
 - ■ Platform Fee Recipient balance increases by 0.4 cUSD (2%).
 - ■ Wallet B (Seller) balance increases by 18.6 cUSD (93%).
 - ■ Wallet C (Buyer) balance decreases by 20 cUSD.
 - ■ Ticket NFT ownership transfers to Wallet C.

Test Case 4.5: Block Self-Purchase

- **Goal:** Prevent sellers from buying back their own tickets.
- **Steps:**
 1. Connect **Wallet B (Seller)** who listed the ticket.
 2. Attempt to call `buyResaleTicket(tokenId, price)`.
- **Expected Result:** ■ Transaction reverts with "Cannot buy your own ticket".

Test Case 4.6: Organizer Resale Purchase Prevention

- **Goal:** Prevent event organizers from buying resale tickets for their own event.
- **Steps:**
 1. Connect **Wallet A (Organizer)** (holds valid KYC identity).
 2. Attempt to purchase a resale ticket for Event ID 0 using `buyResaleTicket`.
- **Expected Result:** ■ Transaction reverts with "Organizer cannot buy resale tickets for their own event".